

Adham Albasha — Training Sessions

Session 1 — Component Composition

Learning Objectives

By the end of this session, the team should understand:

- Why the game screen is split into small components.
- What each component is responsible for.
- How props flow from the page to the components.

1. The Problem

The game screen has many parts: a title, a progress bar, a question, answer options, a submit button, a hint button, a hint box, a clue panel, and status messages. If all of that lived in one file, `GamePage.tsx` would be huge and hard to read.

2. Familiar Idea

In earlier sessions, we broke the Opportunities panel into smaller pieces. The same idea applies here.

3. The New Concept

Component composition means building a screen by combining smaller components. Each component has one job. The page coordinates behavior; the components display.

4. Tiny Example

```
function Header({ title }: { title: string }) {  
  return <h1>{title}</h1>;  
}  
  
function Page() {  
  return <Header title="Welcome" />;  
}
```

5. Project Application

File: `client/src/pages/GamePage.tsx` (composition)

```
<StageHeader title={mystery.title} caseId={mystery.id} />

<StageProgress currentStage={mystery.currentStage} totalStages=
{mystery.totalStages} />

<ProgressBar current={mystery.currentStage} total={mystery.totalStages} />

<form className="answer-form" onSubmit={handleSubmit}>

  <fieldset disabled={pending !== null}>

    <StageQuestion question={mystery.currentQuestion} />

    {mystery.currentOptions.length > 0 ? (

      <AnswerOptions ... />

    ) : (

      <label className="text-answer"> ... </label>

    )}

    <SubmitAnswerButton ... />

  </fieldset>

</form>

<section className="hint-panel">

  <HintButton ... />

  {hint && <HintBox hint={hint} />}

</section>

<CluePanel ... />
```

File: client/src/components/game/StageHeader.tsx

```
interface StageHeaderProps {
  title: string;

  caseId: number;
}

function StageHeader({ title, caseId }: StageHeaderProps) {
  return (
    <div className="stage-header">
      <p className="eyebrow">
        Investigation in progress / File {String(caseId).padStart(3, "0")}
      </p>

      <h1 lang="ar" dir="auto">
        {title}
      </h1>
    </div>
  );
}

export default StageHeader;
```

File: client/src/components/game/StageProgress.tsx

```
interface StageProgressProps {
  currentStage: number;

  totalStages: number;
}

function StageProgress({ currentStage, totalStages }: StageProgressProps) {
  return (
    <div className="stage-progress">
      <p>
        Stage {currentStage + 1} of {totalStages}
      </p>
    </div>
  );
}

export default StageProgress;
```

File: client/src/components/ProgressBar.tsx

```
interface ProgressBarProps {
  current: number;

  total: number;
}

function ProgressBar({ current, total }: ProgressBarProps) {
  const maximum = Math.max(1, total);

  const completed = Math.min(maximum, Math.max(0, current));

  return (
    <progress
      className="progress-bar"
      aria-label="Completed stages"
      value={completed}
      max={maximum}
    />
  );
}

export default ProgressBar;
```

6. Code Walkthrough

StageHeader

What it does: Shows the case file number and title.

Why it exists: The player should know which case they are in.

What you see: "Investigation in progress / File 001" and the Arabic title.

In Plain English: "Heading with a case number and title."

Read it as: "Props in, markup out."

StageProgress

What it does: Shows "Stage X of Y".

Why it exists: The player should know their progress.

What you see: "Stage 1 of 3".

In Plain English: "Add one because the server uses zero-based indexing."

ProgressBar

What it does: Displays a native `<progress>` element.

Why it exists: A visual indicator of progress.

What you see: A thin bar that fills as stages complete.

In Plain English: "Clamp the value to a safe range, then render."

Read it as: "Math.min and Math.max ensure current is between 0 and total."

7. Common Mistakes

WRONG: Putting logic in a presentational component.

`StageProgress` does not need to fetch data. It only displays what it receives.

WRONG: Making one component do too many things.

If a component has more than one clear job, split it.

WRONG: Forgetting that props are read-only.

A component should not modify its props. It should call a callback.

8. Important Decisions

- **Why small components?** Each has one job, easier to read and test.
- **Why props instead of context?** Props are explicit; the parent controls the data.
- **Why `String(caseId).padStart(3, "0")` ?** Formats the ID as `001` .
- **Why `aria-label` on `progress` ?** Screen readers announce the bar's purpose.

9. Project Flow

GamePage fetches mystery

|

▼

Passes mystery.title to StageHeader

|

▼

StageHeader renders the title

|

▼

Passes currentStage and totalStages to StageProgress and ProgressBar

|

▼

Both render their display

10. Review Questions

1. Why split the game screen into small components?
2. What does `StageHeader` receive?
3. What does `ProgressBar` do with its props?
4. Why are props read-only?

11. Glossary

Term	Meaning
------	---------

| Composition | Building a screen from smaller components. |

| Props | Values and functions passed from a parent. |

| Presentational component | A component that only displays data. |

| Callback | A function passed to a component to run later. |

12. What You Should Be Able to Explain Now

- Why the project uses small components.
- What each game component receives.
- How props flow from page to component.

13. Not Yet

- Controlled inputs — Session 2.
 - Result UI and accessibility — Session 3.
-

Session 2 — Controlled Inputs and AnswerOptions

Learning Objectives

By the end of this session, the team should understand:

- What a controlled input is.
- How `AnswerOptions` works.
- Why the selected value lives in the page, not the component.

1. The Problem

The player picks one of four answer options. The page needs to know which one. The radio group needs to reflect the current selection.

2. Familiar Idea

In earlier sessions, the search box was a controlled input. Its value came from state; `onChange` updated state.

The same idea applies here.

3. The New Concept

A **controlled input** is an input whose value is controlled by React state. The input shows what state says; changing it calls a callback that updates state.

4. Tiny Example

```
function NameInput() {  
  const [name, setName] = useState("");  
  
  return <input value={name} onChange={(e) => setName(e.target.value)} />;  
}
```

5. Project Application

File: client/src/components/game/AnswerOptions.tsx

```
interface AnswerOptionsProps {
  options: string[];

  selected: string;

  onSelect: (answer: string) ⇒ void;

  disabled?: boolean;
}

function AnswerOptions({
  options,

  selected,

  onSelect,

  disabled = false,
}: AnswerOptionsProps) {
  return (
    <div className="answer-options">
      {options.map((option, index) ⇒ (
        <label
          className={`answer-option ${
            selected === option ? "is-selected" : ""
          }`}
          key={option}
        >
          <input
            type="radio"
            name="answer"
            value={option}
            checked={selected === option}
            onChange={() ⇒ onSelect(option)}
            disabled={disabled}
          />

          <span className="option-letter" aria-hidden="true">
            {String.fromCharCode(65 + index)}
          </span>

          <span>{option}</span>
        </label>
      ))}
    </div>
  );
};
```

```
}  
  
export default AnswerOptions;
```

File: `client/src/pages/GamePage.tsx` (usage)

```
<AnswerOptions  
  options={mystery.currentOptions}  
  selected={selectedAnswer}  
  onSelect={setSelectedAnswer}  
  disabled={pending === null}  
/>
```

6. Code Walkthrough

The props

```
interface AnswerOptionsProps {  
  options: string[];  
  
  selected: string;  
  
  onSelect: (answer: string) => void;  
  
  disabled?: boolean;  
}
```

What it does: Declares what the component receives.

Why it exists: The component is reusable across mysteries.

What you see: Different options for different stages.

The radio input

```
<input  
  type="radio"  
  name="answer"  
  value={option}  
  checked={selected === option}  
  onChange={() => onSelect(option)}  
  disabled={disabled}  
/>
```

What it does: Renders one radio button per option.

Why it exists: The player must pick exactly one.

What you see: The selected radio is checked.

In Plain English: "If this option matches the selected state, check it."

Read it as: "Controlled radio."

The option letter

```
<span className="option-letter" aria-hidden="true">
  {String.fromCharCode(65 + index)}
</span>
```

What it does: Renders A, B, C, D.

Why it exists: Visual labels for the options.

What you see: A, B, C, D beside the answers.

In Plain English: "65 is A, so index 0 → A, 1 → B, etc."

The `is-selected` class

```
className={`answer-option ${selected === option ? "is-selected" : ""}`}
```

What it does: Adds a CSS class to the selected option.

Why it exists: CSS styles the selected option differently.

What you see: The selected option has a gold border.

7. Common Mistakes

WRONG: Letting the component own the selected value.

```
const [selected, setSelected] = useState(""); // inside AnswerOptions
```

Then the page cannot read the selection.

CORRECT: The page owns the state; the component receives it as a prop.

WRONG: Not setting `checked` on the radio.

Without `checked`, React cannot control the input.

WRONG: Using `defaultChecked` instead of `checked`.

`defaultChecked` makes it uncontrolled. The project uses `checked` .

8. Important Decisions

- **Why `selected` is a prop, not state?** The page needs to know the answer.
- **Why `onSelect` is a callback?** The component reports the change; the page updates state.
- **Why `name="answer"` on all radios?** So they form one group.
- **Why `disabled` prop?** While the answer is being submitted, inputs should be disabled.

9. Project Flow

Player clicks option "brass"

|

▼

onChange fires → onSelect("brass")

|

▼

Page calls setSelectedAnswer("brass")

|

▼

React re-renders

|

▼

AnswerOptions receives selected="brass"

|

▼

The "brass" radio is checked

10. Review Questions

1. What is a controlled input?
2. Why does the page own `selectedAnswer` ?
3. What does `onSelect` do?
4. What does `checked={selected === option}` mean?
5. Why is `name="answer"` on all radios?

11. Glossary

Term	Meaning
------	---------

| Controlled input | An input whose value comes from React state. |

| Callback | A function passed to a component to run later. |

| Radio group | A set of radio inputs with the same `name` . |

| `checked` | The controlled attribute for radio and checkbox inputs. |

12. What You Should Be Able to Explain Now

- What a controlled input is.
- Why the page owns the selected answer.
- How `AnswerOptions` reports changes.
- Why `name` and `checked` matter.

13. Not Yet

- Result UI and accessibility — Session 3.

Session 3 — Result UI and Accessibility

Learning Objectives

By the end of this session, the team should understand:

- What `ResultCard` displays.
- How accessible semantics help all users.
- How the project uses `aria-*` attributes.

1. The Problem

When the mystery is solved, the player deserves a clear ending: the reveal, statistics, and navigation back to the archive. And it should be understandable by all users, including those using assistive tools.

2. Familiar Idea

Earlier sessions used semantic HTML: `<button>`, `<section>`, `<ul>`. The same idea applies here.

3. The New Concept

Accessible semantics means using HTML elements and `aria-*` attributes to describe what an element does. Screen readers and other assistive tools use this information.

4. Tiny Example

```
<button aria-label="Close dialog">x</button>
```

The visual is a small `x`. A screen reader reads "Close dialog".

5. Project Application

File: `client/src/components/game/ResultCard.tsx`

```
import { Link } from "react-router-dom";

interface ResultCardProps {
  id: number;

  title: string;

  reveal: string | null;

  stagesCompleted: number;

  totalStages: number;

  hintsUsed: number;
}

export default function ResultCard({
  id,

  title,

  reveal,

  stagesCompleted,

  totalStages,

  hintsUsed,
}: ResultCardProps) {
  return (
    <div className="solved-seal" aria-hidden="true">
      ✓
    </div>

    <p className="eyebrow">The investigation is complete</p>

    <h1>
      Mystery <em>solved!</em>
    </h1>

    <h2 lang="ar" dir="auto">
      {title}
    </h2>

    <section className="reveal-panel" aria-labelledby="reveal-title">
```

```
    <h2 id="reveal-title">The truth behind the mystery</h2>

    <p lang="ar" dir="rtl">
      {reveal}
    </p>
  </section>

  <dl>
    <dt>Stages completed</dt>

    <dd>
      {stagesCompleted} / {totalStages}
    </dd>

    <dt>Hints used</dt>

    <dd>{hintsUsed}</dd>
  </dl>

  <div className="result-actions">
    <Link className="button" to="/mysteries">
      Explore the archive <span aria-hidden="true">↗</span>
    </Link>

    <Link className="text-link" to={`/${mysteries/${id}}`} >
      View mystery details
    </Link>
  </div>
</>
);
}
```

File: client/src/components/game/CluePanel.tsx

```

<section className="clue-panel" aria-label="Clues">
  <button
    className="clue-toggle"
    type="button"
    onClick={onToggle}
    aria-expanded={isOpen}
    aria-controls="game-clues"
  >
    {isOpen ? "Hide clues" : "Show clues"}

    <span aria-hidden="true">{isOpen ? "-" : "+"}</span>
  </button>

  <div id="game-clues" hidden={!isOpen}>
    ...
  </div>
</section>

```

6. Code Walkthrough

`aria-hidden="true"`

```

<div className="solved-seal" aria-hidden="true">✓</div>

<span aria-hidden="true">↗</span>

```

What it does: Hides the element from screen readers.

Why it exists: The visual is decorative; the meaning is in the surrounding text.

What you see: Nothing changes visually.

`lang` and `dir`

```

<h2 lang="ar" dir="auto">{title}</h2>

<p lang="ar" dir="rtl">{reveal}</p>

```

What it does: Tells assistive tools the content is Arabic.

Why it exists: The stories are in Arabic; the interface is in English.

What you see: Correct text rendering and pronunciation hints.

aria-labelledby

```
<section className="reveal-panel" aria-labelledby="reveal-title">

  <h2 id="reveal-title">The truth behind the mystery</h2>
```

What it does: Names the section using the heading's text.

Why it exists: Screen readers announce "The truth behind the mystery, section".

aria-expanded and **aria-controls**

```
<button

  aria-expanded={isOpen}

  aria-controls="game-clues"

>
```

What it does: Announces whether the controlled region is open.

Why it exists: The clue panel toggles; assistive tools need to know the state.

dl, **dt**, **dd**

```
<dl>
  <dt>Stages completed</dt>

  <dd>
    {stagesCompleted} / {totalStages}
  </dd>

  <dt>Hints used</dt>

  <dd>{hintsUsed}</dd>
</dl>
```

What it does: Renders a definition list.

Why it exists: Semantically correct for name/value pairs.

What you see: Right-aligned values in the styled list.

7. Common Mistakes

WRONG: Using a `<div>` where a semantic element fits.

```
<div onClick={ ... }>Show clues</div> // not a button
```

CORRECT:

```
<button onClick={ ... }>Show clues</button>
```

WRONG: Not marking decorative content as `aria-hidden`.

The  and  should not be announced.

WRONG: Forgetting `lang` on Arabic content.

Screen readers pronounce it incorrectly.

WRONG: Using a bare `×` for a close button without `aria-label`.

A screen reader says "times" instead of "Remove".

8. Important Decisions

- **Why semantic HTML?** It is more accessible by default.
- **Why `aria-hidden` on decorative content?** It prevents confusion.
- **Why `lang` and `dir`?** Correct rendering and pronunciation.
- **Why `d1` instead of `<p>`?** It describes the relationship between label and value.

9. Project Flow

Player solves the final stage

|

▼

Server marks solved: true, reveal: "... "

|

▼

GamePage navigates to /result/:id

|

▼

ResultPage fetches the solved mystery

|

▼

ResultCard renders the reveal, statistics, and navigation

10. Review Questions

1. Why is semantic HTML important?
2. What does `aria-hidden="true"` do?
3. What does `lang="ar"` tell the browser?
4. What does `aria-expanded` announce?
5. Why use `dl` for statistics?

11. Glossary

Term	Meaning
------	---------

| Accessibility | Designing for users with different abilities. |

- | Semantic HTML | Using elements that describe their purpose. |
- | `aria-hidden` | Hides content from assistive tools. |
- | `aria-expanded` | Announces whether a controlled region is open. |
- | `lang` | The language of the content. |
- | `dir` | The text direction (ltr or rtl). |

12. What You Should Be Able to Explain Now

- Why accessibility matters.
- What each `aria-*` attribute does.
- How the project handles Arabic content.
- Why semantic HTML is preferred.

13. Not Yet

- Server-side validation and status codes — Hamid's session.
- Backend structure and mysteries — Hamid's session.

